NOI 模拟题

镜面通道

空间限制：128M　　时间限制：3s

输入输出文件：mirror.in/mirror.out

题目背景:

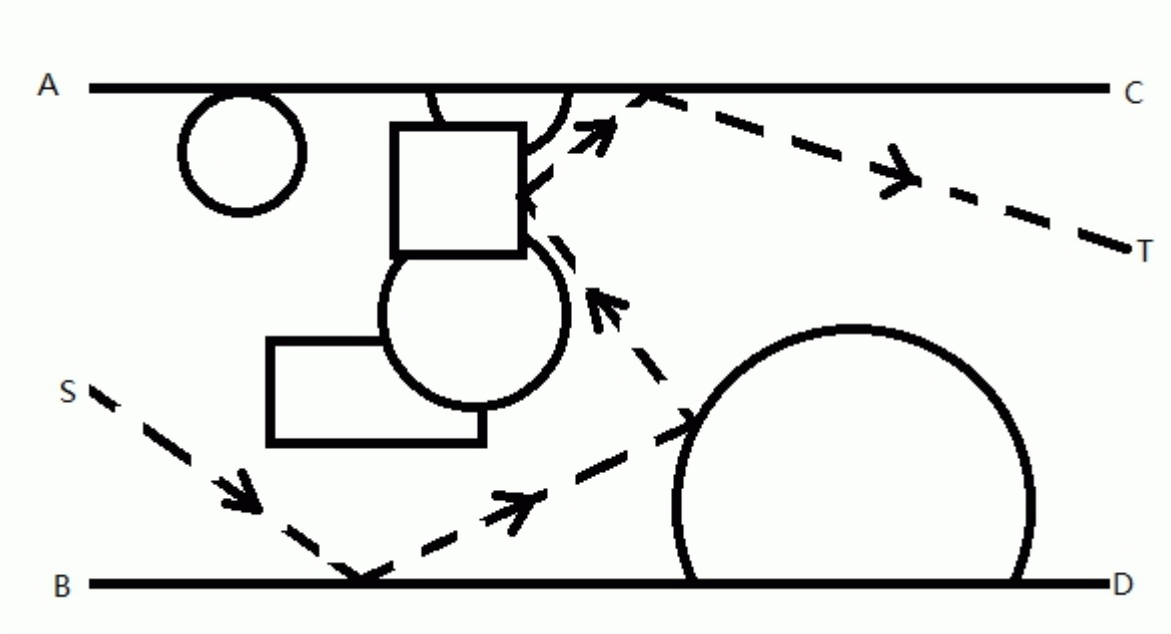
掩体纪元 5 年，星环公司宣布曲率驱动飞船计划，你奉命清理大型激光切割器的镜面通道。

题目描述:

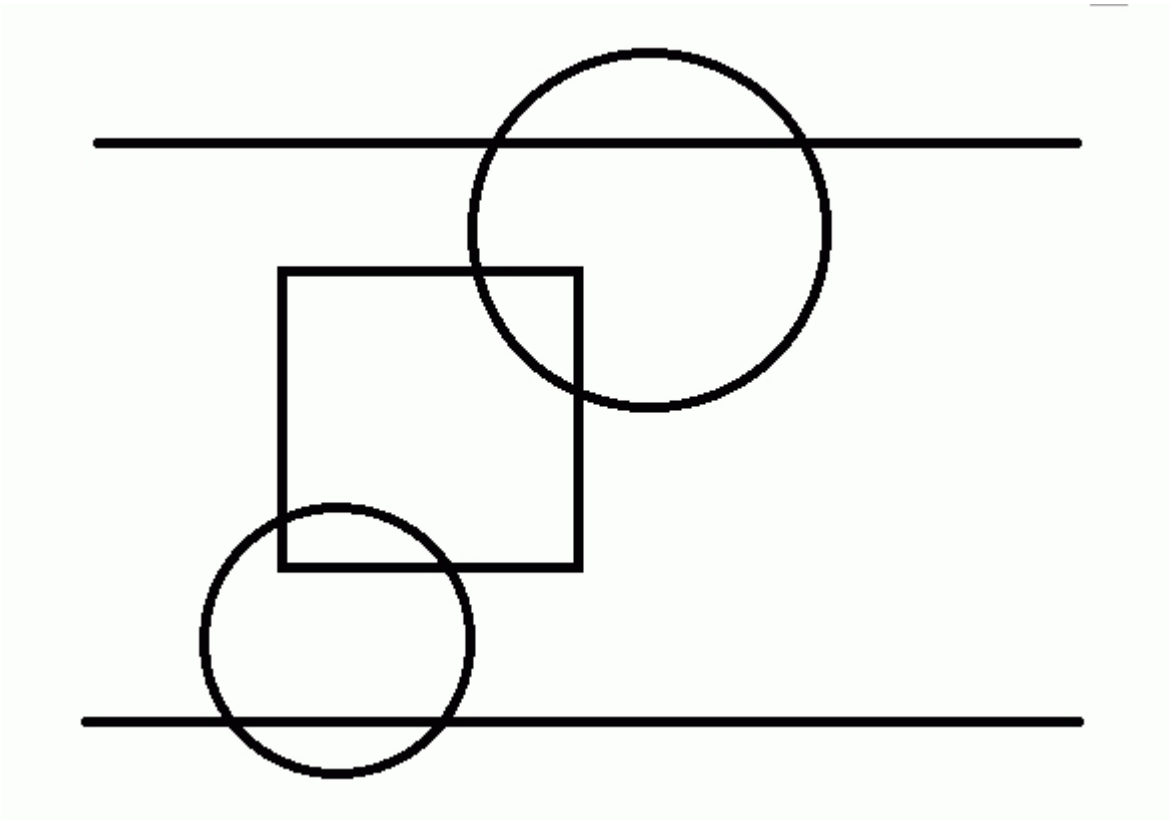
在一个二维平面上，有一个镜面通道，由镜面 AC，BD 组成，AC，BD 长度相等，且都平行于 x 轴，B 位于(0, 0)。

通道中有 n 个外表面为镜面的光学元件，光学元件 α 为圆形，光学元件 β 为矩形(这些元件可以与其他元件和通道有交集，具体看下图)。

光线可以在 AB 上任一点以任意角度射入通道，光线不会发生削弱。当出现元件与元件，元件和通道刚好接触的情况视为光线无法透过(比如两圆相切)。现在给出通道中所有元件的信息(α 元件包括圆心坐标和半径 xi，yi，ri，β 元件包括左下角和右上角坐标 x1，y1，x2，y2)

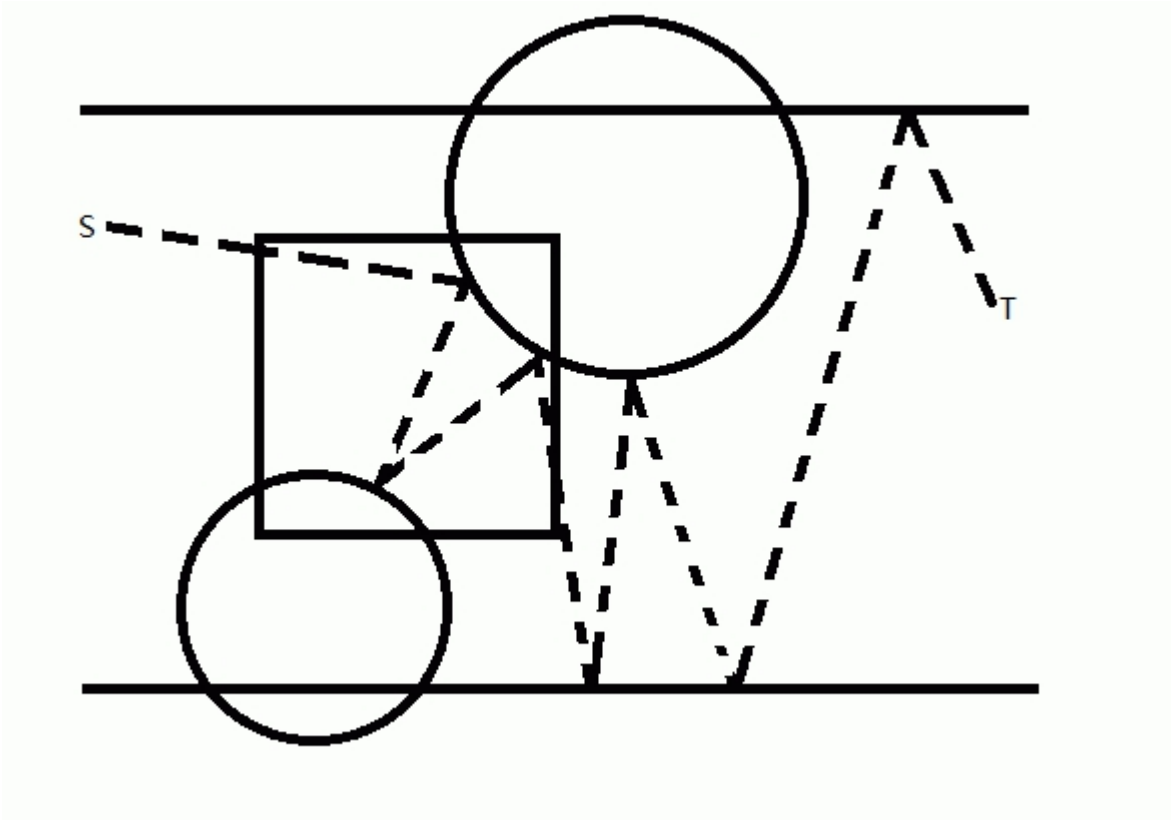


如上图，S 到 T 便是一条合法线路。



当然，显然存在光线无法透过的情况，现在交给你一个艰巨的任务，请求出至少拿走多少个光学元件后，存在一条光线线路可以从 CD 射出。

下面举例说明：



现在假设，取走中间那个矩形，那么就可以构造出一条穿过通道的光路，如图中的 S 到 T。

输入格式:

第一行包含两个整数，x ， y ，表示 C 点坐标。

第二行包含一个整数，n ， 表示有 n 个光学元件。

接下来 n 行:

第一个数字 ti 表示这个元件的类型。

如果 ti = 1，表示元件 α，后面会有三个整数 xi，yi，ri 分别表示圆心坐标和半径。

如果 ti = 2，表示元件 β，后面会有四个整数 x1，y1，x2，y2 分别表示左下角和右上角坐标（矩形都平行，垂直于坐标轴）

输出格式:

输出包含两行:

第一行一个整数，至少需要拿走的光学元件个数 m。

第二行，接下来 m 个数，输出要拿走的光学元件的编号。如果有多组方案输出字典序最小的即可。

样例数据:

mirror.in	mirror.out
1000 100 6 1 500 0 50 2 10 10 20 100 2 100 10 200 100 2 300 10 400 100 2 500 10 600 100 2 700 0 800 100	2 1 6

数据范围及约定:

	N	ti
1	250	1
2	250	1
3	250	1
4	201	1
5	300	1,2
6	232	1,2
7	90	1,2
8	68	1,2
9	300	1,2
10	300	1,2
11	15	2
12	12	2
13	10	2

14	17	2
15	17	1,2
16	10	1,2
17	300	1,2
18	300	1,2
19	300	1,2
20	300	1,2

三体计算机

空间限制: 128M　　时间限制: 1s

输入输出文件: expectation.in/expectation.out

题目背景:

威慑纪元 3 年，社会安定和谐。地球与三体文明的交流不断深入。

作为理论计算机科学家，你开始研究三体计算机。

题目描述:

三体计算机的状态由 01 序列和指针组成。

01 序列从 1 到 n 标号，每个时刻产生一个从 1 到 n 之间随机变量 j，如果此时的指针位置为 i ,则将其移动到 j ,代价为 |i - j|,并且将位置 j 的数取反，如果所有的数字都变成 1 或 0 则停止。

有多组数据，数据组数为 T，对于每组数据，给定序列长度 n，有 m 个询问，每个询问输入一个长度为 n 的 01 序列以及指针的初始位置。

你需要计算三体计算机运行的期望代价。

输出答案 mod 10^9 + 7 的结果。

对于分数,输出分子乘分母的逆元的结果。

输入格式:

第一行一个正整数 T，表示数据组数。

对于每组数据，第一行两个正整数 n， m，为序列长度和询问个数。

接下来 m 行，每行一个长度为 n 的 01 串与初始的指针位置。

输出格式:

对于每个询问，输出一行你的答案。

样例数据 1:

expectation.in	expectation.out
1 2 1 01 1	500000004

样例解释 1:

如果选择 1 号位置，代价为 0，结束操作。

如果选择 2 号位置，代价为 1，结束操作。

两者的可能性相等，所以期望代价为 0.5。

样例数据 2:

expectation.in	expectation.out
1 4 4 1000 1 0010 2 0011 3 1010 4	437500012 62500009 750000015 250000012

数据范围:

数据编号	T	M	N
1	1	1	5
2	1	1	8
3	1	1	10
4	1	5	10
5	5	5	20
6	10	5	20
7	10	10	30
8	10	100	40
9	1	10	100
10	1	100	100

重建计划

输入输出文件: build*.in/build*.out

题目背景:

威慑后纪元，地球防御被击溃，作为联合国粮食计划署执行总干事，你尝试着制作一份修复物资供应的计划。

题目描述:

build1.in ~ build5.in

给定图，选取边集，求最小代价使得关键点点集连通。

build6.in ~ build10.in

给定网格图，选取点集，求最小代价使得关键点点集连通。

点集中选取的点(x,y)可以和(x+1,y),(x,y+1),(x-1,y),(x,y-1)都连通。

输入格式:

build1.in ~ build5.in

第一行包含三个整数 n, m, p。

接下来 m 行,每行三个整数 ui, vi, wi,表示这条边连接的两点和边权。

接下来一行包含 p 个整数,表示关键点的编号。

build6.in ~ build10.in

第一行包含两个个整数 n，m。

接下来 n 行每行 m 个数，第 i 行 j 列表示选取(i,j)的代价。

如果选取(i,j)的代价为 0，表示它为关键点。

输出格式:

build1.out ~ build5.out

第一行一个整数表示选取的边集边权总代价。

第二行一个整数表示选取的边数 s。

接下来 s 行每行一个整数表示选取的边的编号。

build6.out ~ build10.out

第一行一个整数表示选取点集。

接下来 n 行每行一个长度为 m 的字符串。

如果(i,j)为关键点,第 i 行 j 列字符应为“S”

如果(i,j)为被选取的点,第 i 行 j 列字符应为“0”

如果不是以上两种情况,第 i 行 j 列字符应为“X”.

样例数据 1:

build*.in	build*.out
3 3 2 1 2 1 1 3 1 2 3 10	2 2 1 2

2 3	

样例数据 2:

build*.in	build*.out
3 3 0 1 1 1 1 1 1 1 0	3 SOX XOX XOS

计分方式:

如果输出方案不合法，该测试点记 0 分。

如果输出方案合法,记你的答案为 F,标准答案为 T。

那么该测试点你的得分为 min{[10*T/F],12}(其中 [] 表示向下取整)

数据编号	标准答案
1	5652
2	303617
3	429488
4	438417
5	8293
6	42
7	422814142
8	346
9	472167
10	183250

你可以使用选手目录下的 checker 判断自己的答案是否合法。

使用方法为在终端运行命令 ./checker N

你必须保证对应的 build*.in 和 build*.out 在目录下。

ETO is not here!	对应的文件缺失
ETO is invalid!	输出方案格式错误
ETO is schismatical!	输出方案关键点不连通
ETO is great!	输出方案合法